



made by m4n14ck

Introducción al lenguaje Rust

Rust es un lenguaje de programación de sistemas moderno, creado por Mozilla e impulsado actualmente por la **Rust Foundation**. Su principal objetivo es ofrecer el rendimiento de lenguajes como C y C++, pero con un enfoque mucho más seguro, evitando errores comunes relacionados con la gestión de memoria.

Desde su lanzamiento en 2015, Rust ha ganado una gran popularidad gracias a su combinación de **velocidad**, **seguridad** y **conurrencia**, siendo utilizado por empresas como Microsoft, Amazon, Cloudflare, Google y Discord para desarrollar software crítico.

Una de las características más importantes de Rust es su sistema de **ownership (propiedad)**, junto con los conceptos de **borrowing (préstamos)** y **lifetimes (tiempos de vida)**. Estos mecanismos permiten administrar la memoria de forma automática sin necesidad de un recolector de basura (*Garbage Collector*), lo que reduce el consumo de recursos y elimina una gran cantidad de vulnerabilidades relacionadas con el acceso a memoria.

Rust también destaca por ofrecer:

- Alto rendimiento, comparable con C y C++.
- Seguridad en memoria sin sacrificar velocidad.
- Excelente soporte para programación concurrente.
- Un compilador que ayuda a detectar errores antes de ejecutar el programa.
- Un gestor de paquetes integrado llamado **Cargo**, que facilita la creación, compilación y administración de proyectos.

Gracias a estas características, Rust se utiliza en una amplia variedad de áreas, entre ellas:

- Desarrollo de sistemas operativos.
- Aplicaciones de escritorio.
- Servicios web y APIs de alto rendimiento.
- Computación en la nube.
- Desarrollo de videojuegos.
- Sistemas embebidos e Internet de las Cosas (IoT).
- Herramientas de línea de comandos.
- Ciberseguridad y análisis de software.

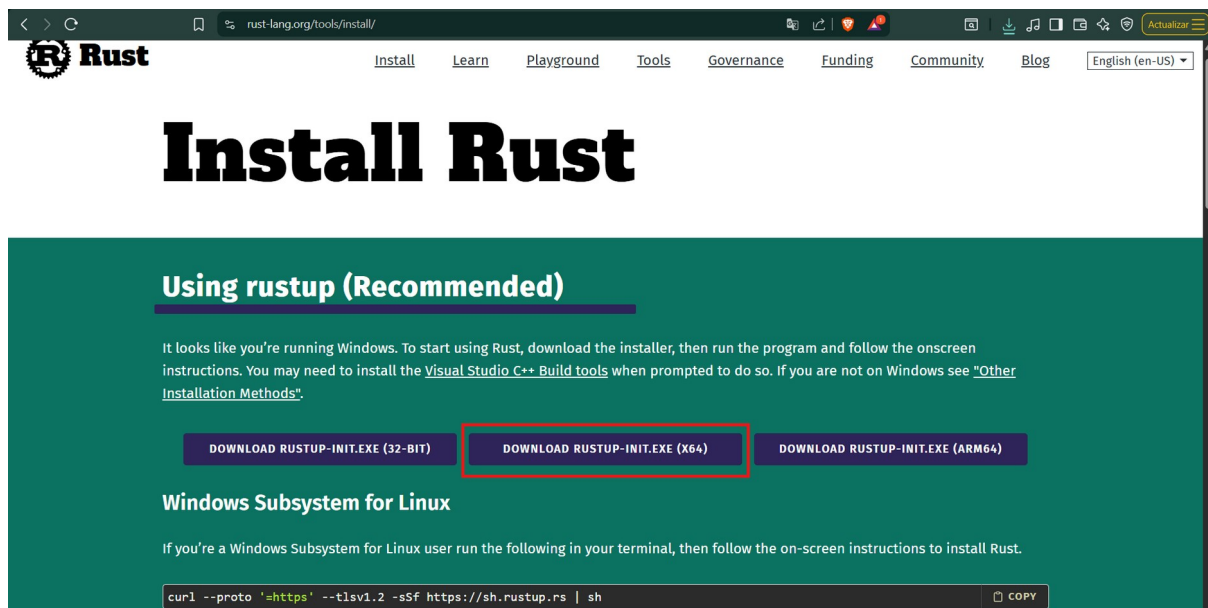
Instalación de Rust en Windows

Requisitos

- Windows 10 o Windows 11 (64 bits).
- Conexión a Internet.
- Permisos para instalar software.

Paso 1. Descargar Rust

1. Abre tu navegador y entra al sitio oficial:
 - <https://www.rust-lang.org/tools/install>



2. Haz clic en **Download rustup-init.exe**.

Paso 2. Ejecutar el instalador

1. Ejecuta el archivo **rustup-init.exe**.
2. Se abrirá una consola con varias opciones.

```
C:\Users\jjb11\Downloads\rus x + v

The Cargo home directory is located at:

C:\Users\jjb11\.cargo

This can be modified with the CARGO_HOME environment variable.

The cargo, rustc, rustup and other commands will be added to
Cargo's bin directory, located at:

C:\Users\jjb11\.cargo\bin

This path will then be added to your PATH environment variable by
modifying the PATH registry key at HKEY_CURRENT_USER\Environment.

You can uninstall at any time with rustup self uninstall and
these changes will be reverted.

Current installation options:

default host triple: x86_64-pc-windows-msvc
default toolchain: stable (default)
profile: default
modify PATH variable: yes

1) Proceed with standard installation (default - just press enter)
2) Customize installation
3) Cancel installation
>1
```

Generalmente basta con presionar: 1

Esto instalará:

- Rust Compiler (rustc)
- Cargo
- Rust Standard Library
- Documentación
- Rustup

Paso 4. Verificar la instalación

Ejecuta en el cmd o powershell: **rustc --version**, **cargo --version**

Si aparece algo similar a:

rustc 1.xx.x

cargo 1.xx.x

```
PS C:\Users\jjb11> rustc --version
rustc 1.97.1 (8bab26f4f 2026-07-14)
PS C:\Users\jjb11> cargo --version
cargo 1.97.1 (c980f4866 2026-06-30)
PS C:\Users\jjb11>
```

Instalación de Rust en Linux

Rust funciona en prácticamente cualquier distribución de Linux.

Paso 1. Instalar dependencias

En Debian, Ubuntu o Linux Mint:

```
sudo apt update  
sudo apt install build-essential curl -y
```

En Fedora:

```
sudo dnf install gcc curl -y
```

En Arch Linux:

```
sudo pacman -S base-devel curl
```

Paso 2. Instalar Rust

Ejecuta el siguiente comando:

```
curl --proto '=https' --tlsv1.2 -sSf https://sh.rustup.rs | sh
```

El instalador mostrará un menú.

```
/home/default/.cargo  
This can be modified with the CARGO_HOME environment variable.  
The cargo, rustc, rustup and other commands will be added to  
Cargo's bin directory, located at:  
  
/home/default/.cargo/bin  
This path will then be added to your PATH environment variable by  
modifying the profile files located at:  
  
/home/default/.profile  
/home/default/.bashrc  
You can uninstall at any time with rustup self uninstall and  
these changes will be reverted.  
Current installation options:  
  
default host triple: x86_64-unknown-linux-gnu  
default toolchain: stable (default)  
profile: default  
modify PATH variable: yes  
1) Proceed with standard installation (default - just press enter)  
2) Customize installation  
3) Cancel installation  
>
```

Selecciona:

1) Proceed with installation (default)

Paso 3. Cargar el entorno

Una vez instalada la herramienta, ejecuta:

```
source $HOME/.cargo/env
```

Paso 4. Verificar la instalación

`rustc --version`

`cargo --version`

```
default@WIN-CU7HAAPVN2L:~$ rustc --version
rustc 1.97.1 (8bab26f4f 2026-07-14)
default@WIN-CU7HAAPVN2L:~$ cargo --version
cargo 1.97.1 (c980f4866 2026-06-30)
default@WIN-CU7HAAPVN2L:~$
```

Actualizar Rust

Para obtener la versión más reciente del lenguaje:

`rustup update`

Desinstalar Rust

Si deseas eliminar Rust del sistema:

`rustup self uninstall`

Cargo: El gestor de proyectos de Rust

¿Qué es Cargo?

Cargo es la herramienta oficial de Rust para la administración de proyectos. Permite crear nuevos proyectos, compilar programas, ejecutar aplicaciones y administrar dependencias de manera sencilla. Al instalar Rust mediante **rustup**, Cargo se instala automáticamente.

Con Cargo, el desarrollador no necesita compilar manualmente cada archivo, ya que esta herramienta automatiza el proceso de construcción del proyecto.

Principales funciones de Cargo

- Crear nuevos proyectos.
- Compilar programas.
- Ejecutar aplicaciones.
- Administrar bibliotecas (dependencias).
- Ejecutar pruebas.
- Generar versiones optimizadas para producción.

Comandos básicos de Cargo

Crear un proyecto nuevo en este caso se usará el WSL(Windows Subsystem for Linux) :

cargo new hola_rust

```
default@WIN-CU7HAAPVN2L:~/rust_programs$ cargo new hola_rust
  Creating binary (application) 'hola_rust' package
note: see more 'Cargo.toml' keys and their definitions at https://doc.rust-lang.org/cargo/reference/manifest.html
default@WIN-CU7HAAPVN2L:~/rust_programs$ ls
hola_rust
default@WIN-CU7HAAPVN2L:~/rust_programs$ |
```

Entrar al proyecto:

cd hola_rust

```
default@WIN-CU7HAAPVN2L:~/rust_programs$ cd hola_rust/
default@WIN-CU7HAAPVN2L:~/rust_programs/hola_rust$ ls
Cargo.toml  src
default@WIN-CU7HAAPVN2L:~/rust_programs/hola_rust$ |
```

Compilar el proyecto:

cargo build

```
default@WIN-CU7HAAPVN2L:~/rust_programs/hola_rust$ cargo build
  Compiling hola_rust v0.1.0 (/home/default/rust_programs/hola_rust)
  Finished `dev` profile [unoptimized + debuginfo] target(s) in 1.29s
default@WIN-CU7HAAPVN2L:~/rust_programs/hola_rust$ |
```

Ejecutar el proyecto:

cargo run

```
default@WIN-CU7HAAPVN2L:~/rust_programs/hola_rust$ cargo run
    Finished `dev` profile [unoptimized + debuginfo] target(s) in 0.01s
    Running `target/debug/hola_rust`
Hello, world!
default@WIN-CU7HAAPVN2L:~/rust_programs/hola_rust$ |
```

Actualizar las dependencias:

cargo update

```
default@WIN-CU7HAAPVN2L:~/rust_programs/hola_rust$ cargo update
    Locking 0 packages to latest Rust 1.97.1 compatible versions
default@WIN-CU7HAAPVN2L:~/rust_programs/hola_rust$ |
```

Cargo.toml contiene la información del proyecto y las dependencias.

Ejemplo:

```
default@WIN-CU7HAAPVN2L:~/rust_programs/hola_rust$ cat Cargo.toml
[package]
name = "hola_rust"
version = "0.1.0"
edition = "2024"

[dependencies]
default@WIN-CU7HAAPVN2L:~/rust_programs/hola_rust$ |
```


Mi primer programa en Rust

Para crear un programa sencillo en Rust desde la terminal, se utilizará Vim, aunque pueden emplear el editor de código que prefieran, o incluso un IDE.

Ahora, antes que nada, vamos a crear una carpeta llamada `capitulo_1`. Después, ingresamos a esa carpeta y dentro de ella crearemos el archivo `1_hola_mundo.rs`.

Ejemplo:

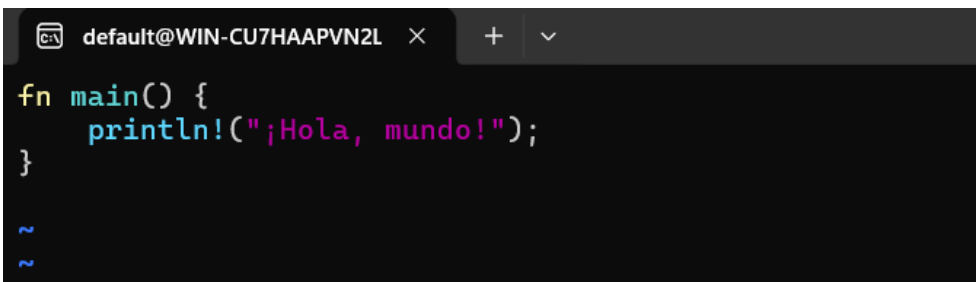
```
mkdir capitulo_1
cd capitulo_1/
```

```
vim 1_hola_mundo.rs|
```

Una vez hecho esto, comenzamos con el desarrollo del programa.

Código del programa

Abre el archivo `1_hola_mundo.rs` con tu editor y escribe lo siguiente:



```
fn main() {
    println!("¡Hola, mundo!");
}

~
~
```

Explicación del código

Explicación de los conceptos nuevos

fn main() {: `fn` declara una función; `main` es el punto de entrada; `()` indica que no recibe parámetros y `{` abre su bloque.

println!("¡Hola, mundo!");: Muestra "¡Hola, mundo!"; `println!` añade un salto de línea.

}: Cierra el bloque actual.

Compilar y ejecutar el programa

Como estamos trabajando con un archivo `.rs` directamente, lo compilamos y ejecutamos con el compilador **rustc**:

```
rustc 1_hola_mundo.rs |
```

Una vez compilado le aparecerá otro archivo con el mismo nombre pero sin la extensión `.rs` ya que este será un ejecutable:

```
default@WIN-CU7HAAPVN2L:~/rust_programs/hola_rust/capitulo_1$ ls  
1_hola_mundo 1_hola_mundo.rs  
default@WIN-CU7HAAPVN2L:~/rust_programs/hola_rust/capitulo_1$ |
```

Para ejecutar el archivo solo debes escribir `./` y el nombre del archivo ejecutable en este caso `1_hola_mundo`.

```
./1_hola_mundo
```

El mensaje se mostrará en la misma terminal

```
¡Hola, mundo!
```

¡Felicidades! Has creado y ejecutado tu primer programa en Rust. En este capítulo aprendiste a:

- Crear una carpeta de proyecto (**capitulo_1**).
- Crear un archivo fuente con extensión **.rs** (**1_hola_mundo.rs**).
- Escribir la estructura básica de un programa en Rust usando **fn main()**.
- Utilizar el macro **println!** para imprimir texto en la consola.
- Compilar y ejecutar el programa con **rustc** o cargo **run**.

Este es el primer paso de tu camino aprendiendo Rust. A partir de aquí, en los siguientes capítulos exploraremos variables, tipos de datos, funciones, control de flujo y mucho más.

Comentarios

Antes de comenzar a escribir programas más complejos, es importante aprender a utilizar los **comentarios**.

Los comentarios son notas que escribimos dentro del código para explicar qué hace una parte del programa. Estos comentarios **no son ejecutados por Rust**, por lo que solo sirven para que el código sea más fácil de entender.

Los comentarios son muy útiles cuando trabajamos en proyectos grandes o cuando regresamos a un programa después de mucho tiempo.

Para esto lo que haremos será crear un nuevo archivo dentro de la carpeta **capitulo_1** el cual este archivo se llamara **2_comentarios.rs**

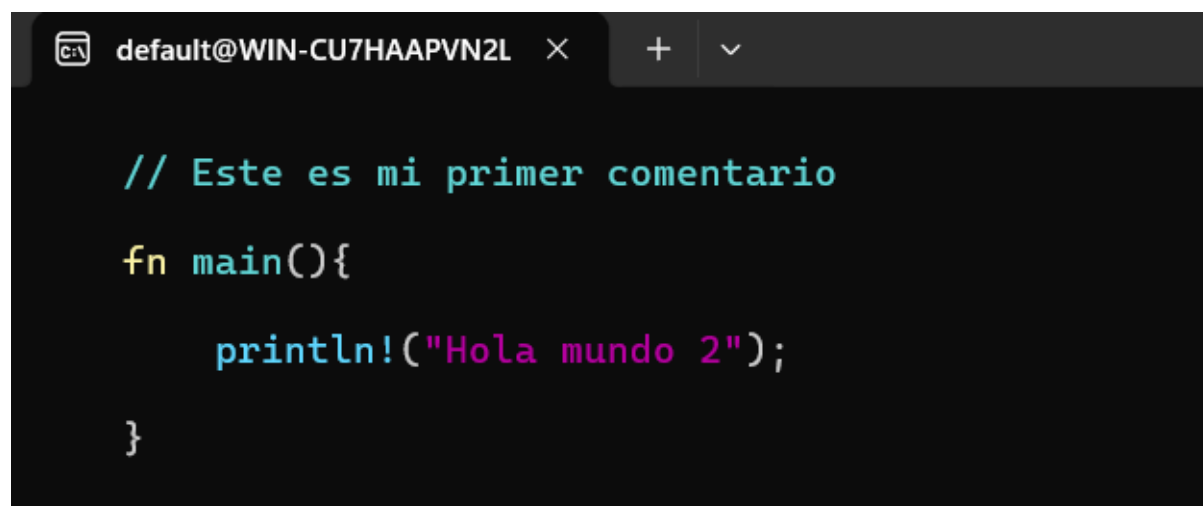
```
2_comentarios.rs|
```

Comentario de una línea

Es el comentario más utilizado.

Se escribe utilizando dos diagonales (`//`).

Escribe el siguiente código en **2_comentarios.rs**.

A screenshot of a code editor window. The title bar shows 'default@WIN-CU7HAAPVN2L' and standard window controls. The code is written in Rust and includes a single-line comment. The code is:

```
// Este es mi primer comentario
fn main(){
    println!("Hola mundo 2");
}
```

Observa que la primera línea comienza con `//`.

Todo lo que esté después de esas dos diagonales será ignorado por el compilador.

Compilar el programa

Abre la terminal dentro de la carpeta **capitulo_1** y ejecuta:

```
rustc 2_comentarios.rs |
```

Ejecutamos:

```
./2_comentarios |
```

La salida será:

```
Hola mundo 2
```

Como puedes observar, el comentario no aparece en la consola porque Rust lo ignora completamente.

Comentarios al final de una línea

También podemos escribir comentarios al final del código.

```
fn main() {  
    println!("Hola Mundo"); // Imprime un saludo
```

Comentarios de varias líneas

Si queremos escribir un comentario más largo, podemos hacerlo de esta forma.

```
/*  
Este programa  
muestra un mensaje  
en pantalla.  
*/  
fn main() {  
    println!("Rust");
```

Explicación de los conceptos nuevos

```
/*: Inicia un comentario de varias líneas.
```

¿Cuándo debemos usar comentarios?

Los comentarios deben explicar **por qué** hacemos algo.

Por ejemplo:

Incorrecto

```
// Imprime Hola Mundo  
println!("Hola Mundo");
```

Explicación de los conceptos nuevos

// **Imprime Hola Mundo:** Los símbolos // convierten el resto de la línea en un comentario que Rust no ejecuta.

Ejercicio

Agrega comentarios al siguiente programa explicando qué hace cada línea.

```
fn main(){  
    println!("Mi nombre es Juan");  
    println!("Estoy aprendiendo Rust");  
}
```

Variables

En todos los programas necesitamos guardar información.

Por ejemplo:

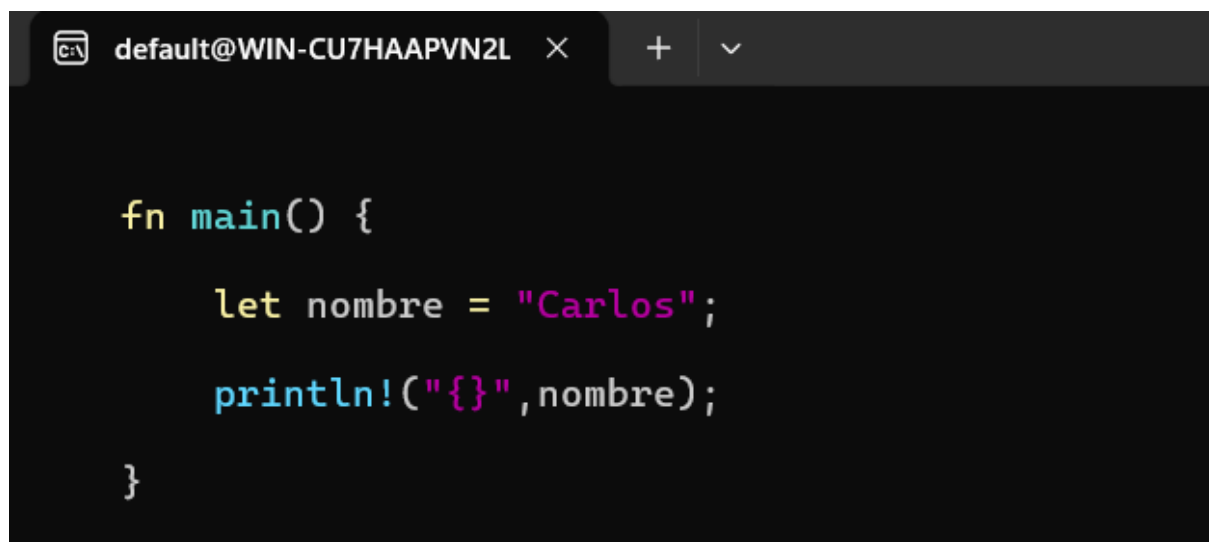
- Un nombre.
- Una edad.
- Un precio.
- Una calificación.

Toda esa información se guarda en **variables**.

Una variable es simplemente un espacio donde almacenamos un dato para poder utilizarlo más adelante.

Para esto lo que haremos será crear un nuevo archivo dentro de la carpeta **capitulo_1** el cual este archivo se llamara **3_variables.rs**

Escribe el siguiente código.



```
fn main() {  
    let nombre = "Carlos";  
    println!("{}", nombre);  
}
```

¿Qué significa cada línea?

`let nombre = "Carlos";`

Explicación de los conceptos nuevos

`let nombre = "Carlos";`: let crea la variable nombre y le asigna "Carlos".

`println!("{}", nombre);`: Muestra "{}"; los marcadores se sustituyen con nombre.

Explicación de los conceptos nuevos

`let edad = 20;`: Crea edad con el valor 20.

Explicación de los conceptos nuevos

```
let precio = 15.75; // También podemos guardar decimales: Crea precio con el valor 15.75.
```

Ejercicio

Crear un programa que guarde:

- Tu nombre.
- Tu edad.
- Tu carrera.

Después imprimir cada uno utilizando `println!`.

Las variables nos permiten guardar información para utilizarla durante la ejecución del programa. En Rust se crean utilizando la palabra `let`, y por defecto son **inmutables**, es decir, su valor no puede cambiar una vez asignado.

Mutabilidad

En Rust las variables son inmutables por defecto. Esto evita cambios accidentales. Cuando necesitamos modificar un valor utilizamos la palabra clave `mut`.

```
fn main() {  
    let mut puntos = 10;  
    puntos = 15;  
    println!("Puntos: {}", puntos);  
}
```

Explicación de los conceptos nuevos

let mut puntos = 10; Crea puntos con el valor 10; puede modificarse.

puntos = 15; Cambia puntos al valor 15.

Constantes

Las constantes se declaran con `const`, deben incluir el tipo y, por convención, su nombre se escribe en mayúsculas. Su valor nunca puede cambiar.

```
const MAX_INTENTOS: u32 = 3;  
  
fn main() {  
    println!("Intentos permitidos: {}", MAX_INTENTOS);  
}
```

Explicación de los conceptos nuevos

const MAX_INTENTOS: u32 = 3;: Declara una constante MAX_INTENTOS, de tipo u32, con el valor fijo 3.

Ejercicio

Crea una variable mutable llamada nivel con valor inicial 1. Cámbiala a 2 y muestra el resultado. Después declara una constante llamada VIDAS con valor 3.

Solución

```
const VIDAS: u8 = 3;

fn main() {
    let mut nivel = 1;
    nivel = 2;

    println!("Nivel: {}", nivel);
    println!("Vidas: {}", VIDAS);
}
```

Compilar y ejecutar

```
rustc 3_variables.rs
./3_variables
```

Salida esperada

```
Nivel: 2
Vidas: 3
```


Lección 4. Tipos de datos

Archivo: `capitulo_1/4_tipos_datos.rs`

Cada valor en Rust tiene un tipo. El compilador normalmente puede inferirlo, pero también podemos escribirlo de manera explícita con dos puntos.

Tipos escalares

- Enteros: `i8`, `i16`, `i32`, `i64`, `i128` e `isize`; también existen `u8`, `u16`, `u32`, `u64`, `u128` y `usize`.
- Decimales: `f32` y `f64`.
- Booleanos: `bool`, cuyos valores son `true` o `false`.
- Caracteres: `char`, escritos entre comillas simples.

```
fn main() {  
    let edad: u8 = 20;  
    let temperatura: f64 = 28.5;  
    let estudiando: bool = true;  
    let inicial: char = 'M';  
  
    println!("Edad: {}", edad);  
    println!("Temperatura: {}", temperatura);  
    println!("¿Estudia Rust?: {}", estudiando);  
    println!("Inicial: {}", inicial);  
}
```

Explicación de los conceptos nuevos

`let edad: u8 = 20;` Crea edad con el valor 20; usa el tipo `u8`.

`let temperatura: f64 = 28.5;` Crea temperatura con el valor 28.5; usa el tipo `f64`.

`let estudiando: bool = true;` Crea estudiando con el valor `true`; usa el tipo `bool`.

`let inicial: char = 'M';` Crea inicial con el valor `'M'`; usa el tipo `char`.

`println!("¿Estudia Rust?: {}", estudiando);` Muestra `"¿Estudia Rust?: {}"`; los marcadores se sustituyen con `estudiando`.

Tuplas y arreglos

Una tupla agrupa valores de tipos diferentes. Un arreglo guarda varios valores del mismo tipo y su tamaño es fijo.

```
fn main() {
    let usuario = ("Ana", 21, true);
    let numeros: [i32; 4] = [10, 20, 30, 40];

    println!("Nombre: {}", usuario.0);
    println!("Segundo número: {}", numeros[1]);
}
```

Explicación de los conceptos nuevos

let usuario = ("Ana", 21, true); Crea usuario con el valor ("Ana", 21, true).

let numeros: [i32; 4] = [10, 20, 30, 40]; Crea numeros con el valor [10, 20, 30, 40]; usa el tipo [i32; 4].

println!("Segundo número: {}", numeros[1]); Muestra "Segundo número: {}"; los marcadores se sustituyen con numeros[1].

Ejercicio

Guarda el nombre de un producto, su precio, la cantidad disponible y si está activo. Muestra todos los valores.

Solución

```
fn main() {
    let producto: &str = "Teclado";
    let precio: f64 = 850.50;
    let cantidad: u16 = 12;
    let activo: bool = true;

    println!("Producto: {}", producto);
    println!("Precio: ${:.2}", precio);
    println!("Cantidad: {}", cantidad);
    println!("Activo: {}", activo);
}
```

Explicación de los conceptos nuevos

let producto: &str = "Teclado"; Crea producto con el valor "Teclado"; usa el tipo &str.

let cantidad: u16 = 12; Crea cantidad con el valor 12; usa el tipo u16.

println!("Precio: \${:.2}", precio); Muestra "Precio: \${:.2}"; presenta el valor con dos decimales.

Compilar y ejecutar

```
rustc 4_tipos_datos.rs
./4_tipos_datos
```

Lección 5. Operadores

Archivo: `capitulo_1/5_operadores.rs`

Los operadores permiten realizar cálculos, comparar valores y combinar condiciones.

Operadores principales

- Aritméticos: `+`, `-`, `*`, `/` y `%`.
- Comparación: `==`, `!=`, `>`, `<`, `>=` y `<=`.
- Lógicos: `&&` (y), `|` (o) y `!` (negación).

```
fn main() {  
    let a = 10;  
    let b = 3;  
  
    println!("Suma: {}", a + b);  
    println!("Residuo: {}", a % b);  
    println!("¿a es mayor?: {}", a > b);  
    println!("Condición: {}", a > 5 && b < 5);  
}
```

Explicación de los conceptos nuevos

`println!("Suma: {}", a + b);`: Muestra "Suma: {}"; los marcadores se sustituyen con `a + b`. `+` realiza una suma.

`println!("Residuo: {}", a % b);`: Muestra "Residuo: {}"; los marcadores se sustituyen con `a % b`. `%` calcula el residuo de una división.

`println!("¿a es mayor?: {}", a > b);`: Muestra "¿a es mayor?: {}"; los marcadores se sustituyen con `a > b`. `>` comprueba si el valor izquierdo es mayor.

`println!("Condición: {}", a > 5 && b < 5);`: Muestra "Condición: {}"; los marcadores se sustituyen con `a > 5 && b < 5`. `&&` exige que las dos condiciones sean verdaderas. `<` comprueba si el valor izquierdo es menor.

Ejercicio

Calcula el área y el perímetro de un rectángulo de base 8.5 y altura 4.0.

Solución

```
fn main() {  
    let base: f64 = 8.5;  
    let altura: f64 = 4.0;  
    let area = base * altura;  
    let perimetro = 2.0 * (base + altura);  
  
    println!("Área: {:.2}", area);  
    println!("Perímetro: {:.2}", perimetro);  
}
```

Explicación de los conceptos nuevos

let area = base * altura; Crea area con el valor base * altura. * realiza una multiplicación.

Compilar y ejecutar

```
rustc 5_operadores.rs  
./5_operadores
```

Salida esperada

```
Área: 34.00  
Perímetro: 25.00
```

Lección 6. Entrada de datos

Archivo: capítulo_1/6_entrada.rs

Para leer datos desde la terminal usamos `std::io::read_line` guarda lo escrito como `String`; `trim` elimina el salto de línea y `parse` convierte el texto al tipo deseado.

```
use std::io::{self, Write};

fn main() {
    print!("Escribe tu edad: ");
    io::stdout().flush().unwrap();

    let mut entrada = String::new();
    io::stdin().read_line(&mut entrada).unwrap();

    let edad: u8 = entrada.trim().parse().unwrap();
    println!("El próximo año tendrás {} años.", edad + 1);
}
```

Explicación de los conceptos nuevos

`use std::io::{self, Write};`: Importa `std::io::{self, Write}`, necesario para utilizar esas herramientas sin escribir la ruta completa.

`print!("Escribe tu edad: ");`: Muestra “Escribe tu edad: ”; `print!` no añade un salto de línea.

`io::stdout().flush().unwrap();`: Ejecuta `io::stdout` con los argumentos indicados.

`let mut entrada = String::new();`: Crea `entrada` con el valor `String::new()`; puede modificarse.

`io::stdin().read_line(&mut entrada).unwrap();`: Ejecuta `io::stdin` con los argumentos indicados.

`let edad: u8 = entrada.trim().parse().unwrap();`: Crea `edad` con el valor `entrada.trim().parse().unwrap()`; usa el tipo `u8`.

Paso a paso

1. Se crea una cadena mutable vacía.
2. `read_line` escribe la entrada del usuario dentro de la cadena.
3. `trim` elimina espacios y el salto de línea final.
4. `parse` convierte el texto; el tipo `u8` indica qué conversión realizar.

Ejercicio

Solicita dos números decimales y muestra su suma, resta, multiplicación y división. Puedes suponer que el segundo número no es cero.

Solución

```
use std::io;

fn leer_numero() -> f64 {
    let mut entrada = String::new();
    io::stdin().read_line(&mut entrada).unwrap();
    entrada.trim().parse().unwrap()
}

fn main() {
    println!("Escribe el primer número:");
    let a = leer_numero();

    println!("Escribe el segundo número:");
    let b = leer_numero();

    println!("Suma: {:.2}", a + b);
    println!("Resta: {:.2}", a - b);
    println!("Multiplicación: {:.2}", a * b);
    println!("División: {:.2}", a / b);
}
```

Explicación de los conceptos nuevos

fn leer_numero() -> f64 {}: Declara la función leer_numero, sin parámetros y devuelve f64.

println!("Resta: {:.2}", a - b);: Muestra "Resta: {:.2}"; presenta el valor con dos decimales. - realiza una resta.

println!("División: {:.2}", a / b);: Muestra "División: {:.2}"; presenta el valor con dos decimales. / realiza una división.

Compilar y ejecutar

```
rustc 6_entrada.rs
./6_entrada
```

Lección 7. Decisiones con if, else y match

Archivo: `capitulo_2/7_decisiones.rs`

Una condición permite ejecutar código diferente según un valor. En Rust no se colocan paréntesis alrededor de la condición.

```
fn main() {
    let edad = 17;

    if edad >= 18 {
        println!("Eres mayor de edad.");
    } else {
        println!("Eres menor de edad.");
    }
}
```

Explicación de los conceptos nuevos

if edad >= 18 {: Comprueba edad >= 18; entra al bloque cuando la condición es verdadera.
>= comprueba si el valor izquierdo es mayor o igual.

} else {: Ejecuta este bloque cuando ninguna condición anterior se cumplió.

Varias condiciones

```
fn main() {
    let calificacion = 85;

    if calificacion >= 90 {
        println!("Excelente");
    } else if calificacion >= 70 {
        println!("Aprobado");
    } else {
        println!("Necesitas estudiar más");
    }
}
```

Explicación de los conceptos nuevos

} else if calificacion >= 70 {: Si la condición anterior falló, comprueba calificacion >= 70.

match

match compara un valor con varios patrones. El guion bajo (`_`) funciona como caso general y cada brazo termina con una coma.

```
fn main() {
    let opcion = 2;

    match opcion {
        1 => println!("Nuevo juego"),
        2 => println!("Cargar partida"),
        3 => println!("Salir"),
        _ => println!("Opción no válida"),
    }
}
```

Explicación de los conceptos nuevos

match opcion {: Compara `opcion` con los patrones de los casos siguientes.

1 => println!("Nuevo juego");: Cuando coincide 1, ejecuta `println!("Nuevo juego")`.

_ => println!("Opción no válida");: Cuando coincide `_`, ejecuta `println!("Opción no válida")`.

Ejercicio

Crea un programa que reciba un número del 1 al 7 y muestre el día correspondiente. Para cualquier otro número debe mostrar “Día no válido”.

Solución

```
fn main() {
    let dia = 5;

    match dia {
        1 => println!("Lunes"),
        2 => println!("Martes"),
        3 => println!("Miércoles"),
        4 => println!("Jueves"),
        5 => println!("Viernes"),
        6 => println!("Sábado"),
        7 => println!("Domingo"),
        _ => println!("Día no válido"),
    }
}
```

Compilar y ejecutar

```
rustc 7_decisiones.rs
./7_decisiones
```

Salida esperada

```
Viernes
```


Lección 8. Bucles

Archivo: `capitulo_2/8_bucles.rs`

Rust ofrece `loop`, `while` y `for` para repetir instrucciones.

loop

```
fn main() {  
    let mut contador = 1;  
  
    loop {  
        println!("{}", contador);  
        contador += 1;  
  
        if contador > 3 {  
            break;  
        }  
    }  
}
```

Explicación de los conceptos nuevos

loop {: Inicia un ciclo indefinido que continuará hasta encontrar `break`.

contador += 1;: Actualiza contador: suma 1 y guarda el resultado.

break;;: Termina el ciclo actual.

while y for

```
fn main() {  
    let mut numero = 3;  
    while numero > 0 {  
        println!("{}", numero);  
        numero -= 1;  
    }  
  
    for valor in 1..=5 {  
        println!("Valor: {}", valor);  
    }  
}
```

Explicación de los conceptos nuevos

while numero > 0 {: Repite el bloque mientras `numero > 0` sea verdadero.

numero -= 1;: Actualiza numero: resta 1 y guarda el resultado.

for valor in 1..=5 {: Recorre `valor` in `1..=5` y ejecuta el bloque una vez por elemento.

Ejercicio

Utiliza un ciclo for para mostrar la tabla de multiplicar del 7, desde 7×1 hasta 7×10 .

Solución

```
fn main() {  
    let numero = 7;  
  
    for i in 1..=10 {  
        println!("{}", numero, i, numero * i);  
    }  
}
```

Compilar y ejecutar

```
rustc 8_bucles.rs  
./8_bucles
```

Lección 9. Funciones

Archivo: `capitulo_2/9_funciones.rs`

Las funciones permiten reutilizar código. Los parámetros deben indicar su tipo. El tipo escrito después de `->` indica el valor que devuelve la función.

```
fn sumar(a: i32, b: i32) -> i32 {
    a + b
}

fn saludar(nombre: &str) {
    println!("Hola, {}", nombre);
}

fn main() {
    saludar("Laura");
    let resultado = sumar(8, 4);
    println!("Resultado: {}", resultado);
}
```

Explicación de los conceptos nuevos

`fn sumar(a: i32, b: i32) -> i32` {}: Declara la función `sumar`, que recibe `a: i32`, `b: i32` y devuelve `i32`.

`fn saludar(nombre: &str)` {}: Declara la función `saludar`, que recibe `nombre: &str`. El símbolo `&` recibe el valor prestado sin tomar su propiedad.

Ejercicio

Crea una función llamada `es_par` que reciba un `i32` y devuelva `true` cuando el número sea par.

Solución

```
fn es_par(numero: i32) -> bool {
    numero % 2 == 0
}

fn main() {
    let numero = 14;

    if es_par(numero) {
        println!("{}", numero);
    } else {
        println!("{}", numero);
    }
}
```

Explicación de los conceptos nuevos

numero % 2 == 0: Esta expresión produce o devuelve `numero % 2 == 0`. `==` compara si dos valores son iguales.

Compilar y ejecutar

```
rustc 9_funciones.rs  
./9_funciones
```

Salida esperada

```
14 es par
```

Lección 10. Ownership y movimiento de valores

Archivo: `capitulo_3/10_ownership.rs`

Ownership significa propiedad. Cada valor tiene una variable propietaria y, cuando esa variable sale de su alcance, Rust libera automáticamente la memoria.

Valores que se copian

Los tipos simples como `i32` implementan Copy, por lo que se puede duplicar su valor.

```
fn main() {
    let a = 10;
    let b = a;

    println!("a = {}, b = {}", a, b);
}
```

Valores que se mueven

String administra memoria dinámica. Al asignarlo a otra variable, la propiedad se mueve para impedir que la misma memoria sea liberada dos veces.

```
fn main() {
    let mensaje = String::from("Hola");
    let copia = mensaje;

    println!("{}", copia);
    // println!("{}", mensaje); // Error: el valor fue movido
}
```

Explicación de los conceptos nuevos

let copia = mensaje; Mueve la propiedad de mensaje hacia copia; después del movimiento, la variable original ya no puede utilizarse.

Clonar cuando necesitamos una copia

```
fn main() {
    let mensaje = String::from("Hola");
    let copia = mensaje.clone();

    println!("{}", mensaje, copia);
}
```

Explicación de los conceptos nuevos

let copia = mensaje.clone(); Crea copia con el valor mensaje.clone(). clone crea una copia independiente y conserva utilizable el valor original.

Ejercicio

Crea un String con el nombre de una ciudad. Clónalo en otra variable y muestra ambos valores.

Solución

```
fn main() {  
    let ciudad = String::from("Veracruz");  
    let respaldo = ciudad.clone();  
  
    println!("Ciudad: {}", ciudad);  
    println!("Respaldo: {}", respaldo);  
}
```

Compilar y ejecutar

```
rustc 10_ownership.rs  
./10_ownership
```

Lección 11. Referencias y borrowing

Archivo: `capitulo_3/11_referencias.rs`

Borrowing significa préstamo. Una referencia (&) permite utilizar un valor sin obtener su propiedad. Así, la variable original continúa siendo válida.

```
fn longitud(texto: &String) -> usize {
    texto.len()
}

fn main() {
    let mensaje = String::from("Rust");
    let cantidad = longitud(&mensaje);

    println!("{}", "tiene {} caracteres", mensaje, cantidad);
}
```

Explicación de los conceptos nuevos

fn longitud(texto: &String) -> usize {}: Declara la función longitud, que recibe texto: &String y devuelve usize. El símbolo & recibe el valor prestado sin tomar su propiedad.

texto.len(): Esta expresión produce o devuelve texto.len().

Referencias mutables

Para modificar un valor prestado usamos &mut y la variable original también debe ser mutable.

```
fn agregar_saludo(texto: &mut String) {
    texto.push_str(", mundo");
}

fn main() {
    let mut mensaje = String::from("Hola");
    agregar_saludo(&mut mensaje);
    println!("{}", mensaje);
}
```

Explicación de los conceptos nuevos

fn agregar_saludo(texto: &mut String) {}: Declara la función agregar_saludo, que recibe texto: &mut String. &mut crea un préstamo mutable, por lo que la función puede cambiar el valor original.

texto.push_str(", mundo");: Ejecuta texto.push_str con los argumentos indicados.

Ejercicio

Crea una función `duplicar` que reciba una referencia mutable a un `i32` y multiplique el valor por 2.

Solución

```
fn duplicar(numero: &mut i32) {  
    *numero *= 2;  
}  
  
fn main() {  
    let mut valor = 12;  
    duplicar(&mut valor);  
    println!("Resultado: {}", valor);  
}
```

Explicación de los conceptos nuevos

`*numero *= 2;`: Accede al valor original mediante `*` y lo multiplica por 2.

Compilar y ejecutar

```
rustc 11_referencias.rs  
./11_referencias
```

Salida esperada

```
Resultado: 24
```


Lección 12. Slices

Archivo: `capitulo_3/12_slices.rs`

Un slice es una referencia a una parte de una colección. No posee los datos y, por eso, no necesita copiarlos. Los slices de texto se escriben como `&str`.

```
fn primera_palabra(texto: &str) -> &str {
    for (i, byte) in texto.bytes().enumerate() {
        if byte == b' ' {
            return &texto[..i];
        }
    }
    texto
}
```

```
fn main() {
    let frase = String::from("Rust seguro y rápido");
    println!("{}", primera_palabra(&frase));
}
```

Explicación de los conceptos nuevos

`for (i, byte) in texto.bytes().enumerate()` {: Recorre `(i, byte) in texto.bytes().enumerate()` y ejecuta el bloque una vez por elemento.

`return &texto[..i];`: Termina la función y devuelve `&texto[..i]`.

Slices de arreglos

```
fn main() {
    let numeros = [10, 20, 30, 40, 50];
    let parte = &numeros[1..4];
    println!("{}", parte);
}
```

Explicación de los conceptos nuevos

`println!("{}", parte);`: Muestra `"{:?}"`; usa el formato de depuración.

Ejercicio

Crea un arreglo con seis números y una función que reciba un slice de `i32` y devuelva su suma.

Solución

```
fn sumar(numeros: &[i32]) -> i32 {
    numeros.iter().sum()
}

fn main() {
    let valores = [2, 4, 6, 8, 10, 12];
    let resultado = sumar(&valores[1..5]);
    println!("Suma: {}", resultado);
}
```

Explicación de los conceptos nuevos

numeros.iter().sum(): Esta expresión produce o devuelve `numeros.iter().sum()`.

Compilar y ejecutar

```
rustc 12_slices.rs
./12_slices
```

Salida esperada

```
Suma: 28
```

Lección 13. Vectores y cadenas String

Archivo: `capitulo_4/13_vectores.rs`

Un `Vec<T>` guarda varios valores del mismo tipo y puede crecer o reducirse durante la ejecución. A diferencia de un arreglo, su tamaño no es fijo.

```
fn main() {
    let mut numeros = vec![10, 20, 30];
    numeros.push(40);
    numeros.remove(0);

    for numero in &numeros {
        println!("{}", numero);
    }
}
```

Explicación de los conceptos nuevos

`let mut numeros = vec![10, 20, 30];`: Crea `numeros` con el valor `vec![10, 20, 30]`; puede modificarse.

`numeros.push(40);`: Ejecuta `numeros.push` con los argumentos indicados.

`numeros.remove(0);`: Ejecuta `numeros.remove` con los argumentos indicados.

Acceso seguro con `get`

```
fn main() {
    let lenguajes = vec!["Rust", "Go", "Python"];

    match lenguajes.get(5) {
        Some(nombre) => println!("{}", nombre),
        None => println!("La posición no existe"),
    }
}
```

Explicación de los conceptos nuevos

`match lenguajes.get(5) {`: Compara `lenguajes.get(5)` con los patrones de los casos siguientes.

`Some(nombre) => println!("{}", nombre);`: Cuando coincide `Some(nombre)`, ejecuta `println!("{}", nombre)`.

`None => println!("La posición no existe");`: Cuando coincide `None`, ejecuta `println!("La posición no existe")`.

Modificar un String

```
fn main() {  
    let mut nombre = String::from("Rust");  
    nombre.push('!');  
    nombre.push_str(" es rápido");  
    println!("{}", nombre);  
}
```

Explicación de los conceptos nuevos

nombre.push_str(" es rápido");: Ejecuta nombre.push_str con los argumentos indicados.

Ejercicio

Crea un vector con cinco calificaciones. Recorre sus referencias, calcula la suma y muestra el promedio.

Solución

```
fn main() {  
    let calificaciones = vec![90.0, 85.0, 78.0, 95.0, 88.0];  
    let mut suma = 0.0;  
  
    for nota in &calificaciones {  
        suma += nota;  
    }  
  
    let promedio = suma / calificaciones.len() as f64;  
    println!("Promedio: {:.2}", promedio);  
}
```

Compilar y ejecutar

```
rustc 13_vectores.rs  
./13_vectores
```

Salida esperada

```
Promedio: 87.20
```

Lección 14. Structs

Archivo: `capitulo_4/14_structs.rs`

Una struct define un tipo propio con varios campos relacionados. Es útil para representar usuarios, productos, vehículos y otros elementos del programa.

```
struct Usuario {
    nombre: String,
    edad: u8,
    activo: bool,
}

fn main() {
    let usuario = Usuario {
        nombre: String::from("Elena"),
        edad: 22,
        activo: true,
    };

    println!("{}", usuario.nombre, usuario.edad);
    println!("Activo: {}", usuario.activo);
}
```

Explicación de los conceptos nuevos

struct Usuario {: Define la estructura Usuario, que agrupará los campos siguientes.

nombre: String,: El campo nombre almacenará un valor de tipo String.

}; Cierra la estructura actual.

Métodos con impl

```
struct Rectangulo {
    ancho: f64,
    alto: f64,
}

impl Rectangulo {
    fn area(&self) -> f64 {
        self.ancho * self.alto
    }
}

fn main() {
    let figura = Rectangulo { ancho: 8.0, alto: 3.5 };
    println!("Área: {:.2}", figura.area());
}
```

Explicación de los conceptos nuevos

impl Rectangulo {: Inicia la implementación de métodos para Rectangulo.

println!("Área: {:.2}", figura.area());: Muestra "Área: {:.2}"; presenta el valor con dos decimales.

Ejercicio

Define una struct Producto con nombre, precio y existencias. Agrega un método disponible que devuelva true cuando existencias sea mayor que cero.

Solución

```
struct Producto {
    nombre: String,
    precio: f64,
    existencias: u32,
}

impl Producto {
    fn disponible(&self) -> bool {
        self.existencias > 0
    }
}

fn main() {
    let producto = Producto {
        nombre: String::from("Mouse"),
        precio: 450.0,
        existencias: 8,
    };

    println!("Producto: {}", producto.nombre);
    println!("Precio: ${:.2}", producto.precio);
    println!("Disponible: {}", producto.disponible());
}
```

Explicación de los conceptos nuevos

println!("Disponible: {}", producto.disponible());: Muestra "Disponible: {}"; los marcadores se sustituyen con producto.disponible().

Compilar y ejecutar

```
rustc 14_structs.rs
./14_structs
```

Lección 15. Enums, Option y match

Archivo: `capitulo_4/15_enums.rs`

Una enum representa un valor que puede pertenecer a una de varias variantes. `match` obliga a considerar todas las posibilidades.

```
enum Estado {
    Pendiente,
    EnProceso,
    Terminado,
}

fn mostrar(estado: Estado) {
    match estado {
        Estado::Pendiente => println!("Pendiente"),
        Estado::EnProceso => println!("En proceso"),
        Estado::Terminado => println!("Terminado"),
    }
}

fn main() {
    mostrar(Estado::EnProceso);
}
```

Explicación de los conceptos nuevos

enum Estado {: Define el enum Estado con varias opciones posibles.

Pendiente,: Pendiente es una de las variantes posibles.

Option<T>

Rust no utiliza null como muchos otros lenguajes. `Option<T>` expresa que puede existir un valor con `Some(valor)` o no existir con `None`.

```
fn buscar_par(numeros: &[i32]) -> Option<i32> {
    for numero in numeros {
        if numero % 2 == 0 {
            return Some(*numero);
        }
    }
    None
}

fn main() {
    match buscar_par(&[1, 3, 8, 9]) {
        Some(numero) => println!("Par encontrado: {}", numero),
        None => println!("No se encontró un número par"),
    }
}
```

Explicación de los conceptos nuevos

fn buscar_par(numeros: &[i32]) -> Option<i32> {: Declara la función `buscar_par`, que recibe `numeros: &[i32]` y devuelve `Option<i32>`. El símbolo `&` recibe el valor prestado sin tomar su propiedad. `Option` expresa que puede devolver `Some(valor)` o `None`.

Ejercicio

Crea una enum `Nivel` con las variantes `Bajo`, `Medio` y `Alto`. Escribe una función que reciba un `Nivel` y muestre una descripción.

Solución

```
enum Nivel {
    Bajo,
    Medio,
    Alto,
}

fn describir(nivel: Nivel) {
    match nivel {
        Nivel::Bajo => println!("Riesgo bajo"),
        Nivel::Medio => println!("Riesgo moderado"),
        Nivel::Alto => println!("Riesgo alto"),
    }
}

fn main() {
    describir(Nivel::Alto);
}
```

Compilar y ejecutar

```
rustc 15_enums.rs
./15_enums
```

Salida esperada

```
Riesgo alto
```


Lección 16. Manejo de errores con Result

Archivo: `capitulo_5/16_errores.rs`

Las operaciones que pueden fallar suelen devolver `Result<T, E>`. `Ok` contiene el resultado correcto y `Err` contiene información sobre el error.

Manejar un resultado con match

```
use std::fs;

fn main() {
    let resultado = fs::read_to_string("mensaje.txt");

    match resultado {
        Ok(contenido) => println!("{}", contenido),
        Err(error) => println!("No se pudo leer el archivo: {}", error),
    }
}
```

Explicación de los conceptos nuevos

`Ok(contenido) => println!("{}", contenido);`: Cuando coincide `Ok(contenido)`, ejecuta `println!("{}", contenido)`.

`Err(error) => println!("No se pudo leer el archivo: {}", error);`: Cuando coincide `Err(error)`, ejecuta `println!("No se pudo leer el archivo: {}", error)`.

Propagar errores con ?

El operador `?` devuelve el error automáticamente a la función que llamó. Solo puede usarse en una función cuyo retorno sea compatible con `Result`.

```
use std::fs;
use std::io;

fn leer_archivo(ruta: &str) -> Result<String, io::Error> {
    let contenido = fs::read_to_string(ruta)?;
    Ok(contenido)
}

fn main() {
    match leer_archivo("mensaje.txt") {
        Ok(texto) => println!("{}", texto),
        Err(error) => println!("Error: {}", error),
    }
}
```

Explicación de los conceptos nuevos

fn leer_archivo(ruta: &str) -> Result<String, io::Error> {: Declara la función leer_archivo, que recibe ruta: &str y devuelve Result<String, io::Error>. El símbolo & recibe el valor prestado sin tomar su propiedad. Result distingue entre un resultado correcto, Ok, y un error, Err.

let contenido = fs::read_to_string(ruta)?; Crea contenido con el valor fs::read_to_string(ruta)?. El operador ? extrae el valor si todo salió bien o propaga el error inmediatamente.

unwrap y expect

unwrap obtiene el valor correcto, pero termina el programa si hay un error. expect hace lo mismo y permite añadir un mensaje. Son cómodos para ejercicios, pero en programas reales suele ser mejor manejar el error.

Ejercicio

Solicita un número. Si la conversión es correcta, muestra su doble; si falla, muestra un mensaje sin cerrar el programa de manera inesperada.

Solución

```
use std::io;

fn main() {
    println!("Escribe un número entero:");

    let mut entrada = String::new();
    io::stdin().read_line(&mut entrada).unwrap();

    match entrada.trim().parse::<i32>() {
        Ok(numero) => println!("El doble es {}", numero * 2),
        Err(_) => println!("La entrada no es un número entero válido"),
    }
}
```

Compilar y ejecutar

```
rustc 16_errores.rs
./16_errores
```

Lección 17. Proyecto final: gestor de tareas

Archivo: `capitulo_5/17_proyecto_final.rs`

Este proyecto reúne variables, entrada de datos, vectores, structs, enums, match, funciones, referencias y manejo básico de errores. El programa permite agregar, listar y completar tareas mientras se encuentra en ejecución.

Código completo

```
use std::io::{self, Write};

#[derive(Debug)]
enum Estado {
    Pendiente,
    Completada,
}

struct Tarea {
    descripcion: String,
    estado: Estado,
}

fn leer_linea(mensaje: &str) -> String {
    print!("{}", mensaje);
    io::stdout().flush().unwrap();

    let mut entrada = String::new();
    io::stdin().read_line(&mut entrada).unwrap();
    entrada.trim().to_string()
}

fn listar(tareas: &[Tarea]) {
    if tareas.is_empty() {
        println!("No hay tareas.");
        return;
    }

    for (indice, tarea) in tareas.iter().enumerate() {
        println!("{}", indice + 1, tarea.descripcion, tarea.estado);
    }
}

fn completar(tareas: &mut [Tarea], posicion: usize) -> Result<(), String> {
    match tareas.get_mut(posicion) {
        Some(tarea) => {
            tarea.estado = Estado::Completada;
            Ok(())
        }
        None => Err(String::from("La tarea no existe")),
    }
}
```

```

}

fn main() {
    let mut tareas: Vec<Tarea> = Vec::new();

    loop {
        println!("\n1. Agregar tarea");
        println!("2. Listar tareas");
        println!("3. Completar tarea");
        println!("4. Salir");

        let opcion = leer_linea("Selecciona una opción: ");

        match opcion.as_str() {
            "1" => {
                let descripcion = leer_linea("Descripción: ");
                tareas.push(Tarea {
                    descripcion,
                    estado: Estado::Pendiente,
                });
            }
            "2" => listar(&tareas),
            "3" => {
                listar(&tareas);
                let entrada = leer_linea("Número de tarea: ");

                match entrada.parse::<usize>() {
                    Ok(numero) if numero > 0 => {
                        if let Err(error) = completar(&mut tareas, numero - 1) {
                            println!("{}", error);
                        }
                    }
                    _ => println!("Número no válido"),
                }
            }
            "4" => {
                println!("Programa finalizado.");
                break;
            }
            _ => println!("Opción no válida"),
        }
    }
}

```

Explicación de los conceptos nuevos

`#[derive(Debug)]`: Pide al compilador implementar automáticamente `Debug` para el tipo siguiente.

`entrada.trim().to_string()`: Esta expresión produce o devuelve `entrada.trim().to_string()`.

if tareas.is_empty() {: Comprueba `tareas.is_empty()`; entra al bloque cuando la condición es verdadera.

for (indice, tarea) in tareas.iter().enumerate() {: Recorre `(indice, tarea)` in `tareas.iter().enumerate()` y ejecuta el bloque una vez por elemento.

match tareas.get_mut(posicion) {: Compara `tareas.get_mut(posicion)` con los patrones de los casos siguientes.

let mut tareas: Vec<Tarea> = Vec::new();: Crea tareas con el valor `Vec::new()`; puede modificarse y usa el tipo `Vec<Tarea>`.

match opcion.as_str() {: Compara `opcion.as_str()` con los patrones de los casos siguientes.

Ok(numero) if numero > 0 => {: Cuando coincide `Ok(numero) if numero > 0`, ejecuta `{`.

if let Err(error) = completar(&mut tareas, numero - 1) {: Ejecuta el bloque únicamente si el resultado coincide con el patrón indicado.

Reto de mejora

Agrega una opción para eliminar tareas y otra para mostrar únicamente las tareas pendientes. Después intenta guardar las tareas en un archivo de texto.

Resumen del curso

Al completar estas lecciones aprendiste a:

- Crear, compilar y ejecutar archivos .rs con rustc.
- Declarar variables, constantes y tipos de datos.
- Leer información desde la terminal y utilizar operadores.
- Controlar el flujo con if, else, match y bucles.
- Organizar código mediante funciones.
- Comprender ownership, movimientos, referencias, borrowing y slices.
- Trabajar con vectores, cadenas, structs y enums.
- Representar valores opcionales con Option y manejar fallos con Result.
- Combinar los fundamentos en un programa completo.

Siguiente paso: Practica modificando cada ejemplo. Después podrás estudiar Cargo con dependencias, módulos, traits, genéricos, iteradores, closures y pruebas.